

PakBazer

A Pakistani E-Commerce Web Platform
Project Report

Program: BS Software Engineering (BSSE)

Course: Object-Oriented Programming

Batch:2022-2026

Submitted by:

Ahmed Ali Ghorī — 60117

Komal Raza — 63424

Instructor: Usama Ahmed Hashmi

Table of Contents

- *Project Title*
- *Introduction*
- *Problem Statement*
- *Objectives*
- *Literature Review*
- *System Analysis and Design*
- *Methodology*
- *Implementation Details*
- *Results and Testing*
- *Conclusion and Future Work*
- *References*
- *Appendices*

Project Title

PakBzer: A Pakistani E-Commerce Web Application

The project is a Java-based online fashion store built to demonstrate Object-Oriented Programming principles in a practical full-stack web application.

Introduction

PakBzer is a Spring Boot e-commerce application designed for Pakistani clothing and fashion products. It allows visitors to browse products by category, search for items, view product details, add products to a session cart, register or sign in, write product reviews, and proceed to checkout.

The system is intentionally structured as a layered OOP application. Controllers handle web requests, services contain business rules, repositories provide persistence access, and model classes represent the domain. This structure makes the project suitable for an OOP final assignment because the code demonstrates encapsulation, inheritance, abstraction, polymorphism, composition, and association in one coherent application.

Problem Statement

Small fashion retailers need a clear online shopping experience where customers can discover products, evaluate details and reviews, manage a cart, and complete checkout without manual coordination. At the same time, an OOP course project must prove that object-oriented concepts are not only theoretical definitions but useful design tools for maintainable software.

The problem addressed by PakBzer is therefore twofold: to model a practical e-commerce workflow for a local fashion store and to implement that workflow through a clean object-oriented design that is easy to understand, test, and extend.

Objectives

- *Develop a working online store with category browsing, product detail pages, search, cart management, authentication, reviews, and checkout flow.*
- *Represent real-world store concepts through Java classes such as Product, User, Review, Cart, and CartItem.*
- *Demonstrate OOP pillars: encapsulation, inheritance, abstraction, and polymorphism.*
- *Use a layered Spring MVC architecture so presentation, business logic, data access, and persistence stay separated.*
- *Persist product, user, and review data through Spring Data JPA and an H2 file database.*
- *Provide a responsive user interface using Thymeleaf templates and Bootstrap styling.*
- *Verify the application through Maven build checks and manual browser testing of the main user flows.*

Literature Review

Modern web applications commonly use an MVC or layered architecture to reduce coupling between screens, business rules, and persistence. Spring Boot supports this pattern by providing embedded server support, auto-configuration, Spring MVC, dependency injection, and build integration for Java applications [1].

For security, Spring Security provides authentication and access-control features that integrate with servlet-based Spring applications [2]. PakBzer uses this foundation for form login and Google OAuth2 login. Thymeleaf is used as a server-side template engine that works naturally with Spring MVC and allows dynamic HTML rendering while keeping templates readable [3].

Spring Data JPA helps implement repository-style data access by generating queries from repository interfaces. H2 provides a lightweight Java SQL database suitable for local development and academic demonstration [4]. Stripe Checkout is used as a payment integration concept, while Bootstrap provides responsive interface components for product cards, forms, navigation, and layout [5][6].

System Analysis and Design

Functional Requirements

Requirement	Description
Product catalog	Display products across Men, Women, Kids, Boy, and Adult categories.
Search	Find products by name, description, or brand.
Product detail	Show product image, brand, stock status, rating, price, description, and reviews.
Cart	Add, update, remove, and total items in a session-scoped shopping cart.
Authentication	Support account registration, form login, logout, and Google OAuth2 login.
Reviews	Allow authenticated users to submit ratings and comments for products.
Checkout	Start Stripe Checkout when configured or show a demo checkout flow.

Table 1. Functional requirements.

Non-Functional Requirements

- *Maintainability:* separate controllers, services, repositories, domain models, security, and configuration.
- *Usability:* responsive Bootstrap interface with product cards, category tiles, forms, and feedback messages.
- *Security:* password hashing through BCrypt and route protection for checkout/review workflows.
- *Extensibility:* service interfaces and entity relationships allow future payment methods, products, or admin features.
- *Local deployability:* Maven, embedded Tomcat, and H2 allow the project to run without external database setup.

Architecture Diagram

PakBzer Layered Architecture

Spring MVC separates user interface, business logic, data access, and persistence.

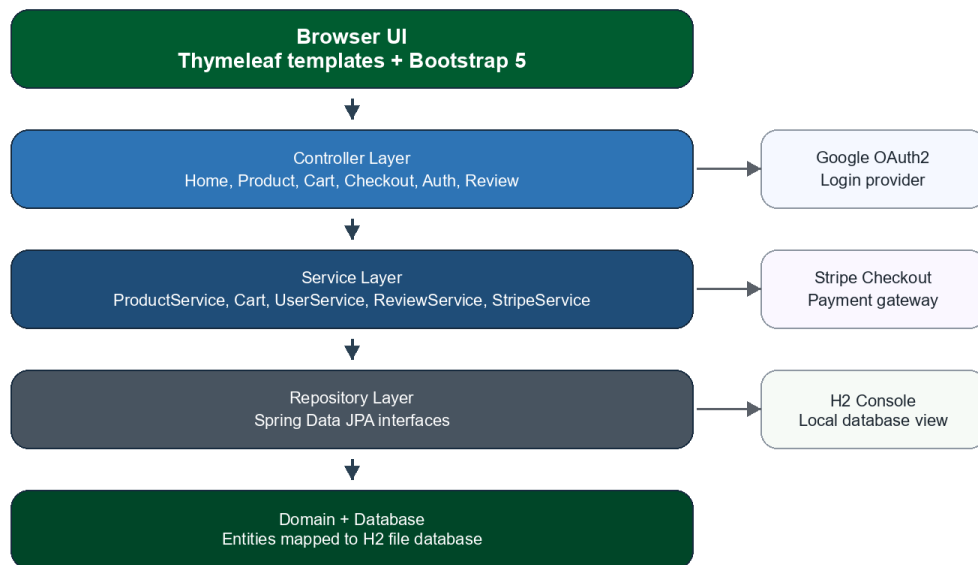


Figure 1. PakBzer layered architecture.

OOP Class Design

The class structure places common persistence fields in an abstract superclass, keeps domain behavior inside model and service classes, and uses interfaces where controller code should depend on a contract rather than a concrete class.

OOP Design Map

Main classes and relationships used to demonstrate OOP principles.

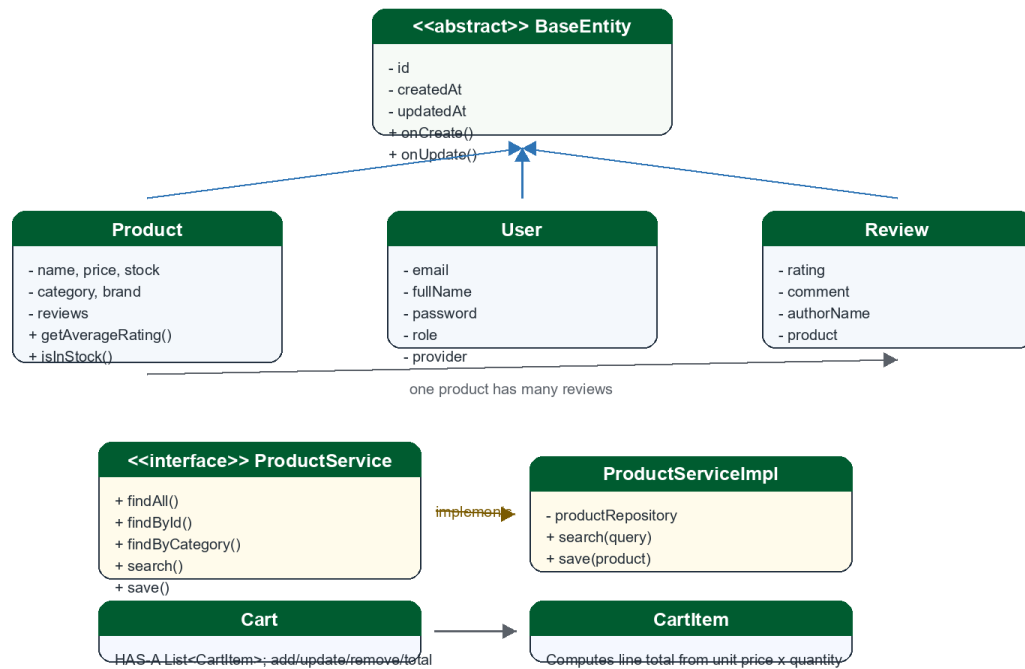


Figure 2. OOP design map showing inheritance, interface implementation, association, and composition.

Main Entity Relationships

Entity / Class	Relationship	Purpose
Product -> BaseEntity	Inheritance	Product receives id and audit timestamps.
User -> BaseEntity	Inheritance	User receives shared persistence metadata.
Review -> BaseEntity	Inheritance	Review receives shared persistence metadata.
Product -> Review	One-to-many association	A product can have many customer reviews.
Review -> Product	Many-to-one association	Each review belongs to one product.
Cart -> CartItem	Composition	A session cart owns cart items and total calculations.

<i>ProductService -> ProductServiceImpl</i>	<i>Interface polymorphism</i>	<i>Controllers depend on the service contract.</i>
--	-------------------------------	--

Table 2. Major object relationships.

Methodology

The project was developed using an iterative object-oriented methodology. First, the domain objects were identified from the problem: product, category, user, review, cart item, and cart. Next, responsibilities were assigned to layers so that each class had a focused role. The application was then implemented with Spring Boot, tested locally, and refined through browser-based verification.

1. Analysis: define store features, user workflows, and OOP learning outcomes.
2. Design: map domain classes, service interfaces, repository contracts, and MVC routes.
3. Implementation: build Java models, controllers, services, repositories, security, templates, and CSS.
4. Data setup: seed sample Pakistani fashion products and reviews through a CommandLineRunner.
5. Verification: run Maven build checks and manually test home, category, product, cart, search, and login flows.

Implementation Details

Technology Stack

Layer	Technology
Language	Java 17
Backend framework	Spring Boot 3.2.5
Web/MVC	Spring MVC controllers and Thymeleaf views
Persistence	Spring Data JPA with H2 file database
Security	Spring Security, BCrypt, Google OAuth2 client
Payments	Stripe Java SDK / demo fallback
Frontend	Bootstrap 5, Bootstrap Icons, custom CSS
Build tool	Apache Maven

Table 3. Project technology stack.

Project Structure

```
src/main/java/com/pakbzer
|-- PakBzerApplication.java
|-- controller/  HomeController, ProductController, CartController, AuthController
|-- model/       BaseEntity, Product, User, Review, CartItem, Category, Role
|-- repository/  ProductRepository, UserRepository, ReviewRepository
|-- service/     ProductService, ProductServiceImpl, Cart, UserService, ReviewService
|-- security/    SecurityConfig, CustomUserDetailsService, CustomOidcUserService
|-- config/      DataSeeder, PasswordConfig, StripeProperties

src/main/resources
|-- templates/   Thymeleaf pages and fragments
|-- static/css/  Custom visual styling
|-- application.properties
```

OOP Pillars in the Code

Inheritance and Auditing

The abstract `BaseEntity` centralizes `id` and `timestamp` fields. `Product`, `User`, and `Review` reuse the same persistence structure by extending this class.

```
@MappedSuperclass
public abstract class BaseEntity {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private LocalDateTime createdAt;
    private LocalDateTime updatedAt;

    @PrePersist
    protected void onCreate() {
        this.createdAt = LocalDateTime.now();
        this.updatedAt = this.createdAt;
    }
}
```

Encapsulation

`Product` keeps review and stock rules inside the class, so callers do not need to duplicate rating or stock calculations.

```
public double getAverageRating() {
    if (reviews == null || reviews.isEmpty()) {
        return 0.0;
    }
    return reviews.stream().mapToInt(Review::getRating).average().orElse(0.0);
}

public boolean isInStock() {
    return stock > 0;
}
```

Abstraction and Polymorphism

`ProductService` defines the product catalog contract. `ProductServiceImpl` provides the current implementation, and controllers can depend on the interface.

```
public interface ProductService {
    List<Product> findAll();
    Optional<Product> findById(Long id);
    List<Product> findByCategory(Category category);
    List<Product> search(String query);
    Product save(Product product);
}
```



```

@Service
public class ProductServiceImpl implements ProductService {
    @Override
    public List<Product> search(String query) {
        if (query == null || query.isBlank()) return Collections.emptyList();
        String q = query.trim();
        return productRepository
            .findByNameContainingIgnoreCaseOrDescriptionContainingIgnoreCaseOrBrandContainingIgnoreCase(q, q, q);
    }
}

```

Composition in the Cart

The session-scoped Cart object owns a collection of CartItem objects. It encapsulates add, update, remove, clear, total item count, and total price behavior.

```

@Component
@Scope(value = WebApplicationContext.SCOPE_SESSION, proxyMode = ScopedProxyMode.TARGET_CLASS)
public class Cart implements Serializable {
    private final List<CartItem> items = new ArrayList<>();

    public synchronized void add(Product product, int quantity) {
        for (CartItem item : items) {
            if (item.getProduct().equals(product)) {
                item.increaseQuantity(quantity);
                return;
            }
        }
        items.add(new CartItem(product.getId(), product.getName(),
            product.getImageUrl(), product.getPrice(), quantity));
    }
}

```

Screenshots



Figure 3. Home page with category tiles and product previews.



Figure 4. Product detail page with rating, stock, and add-to-cart control.

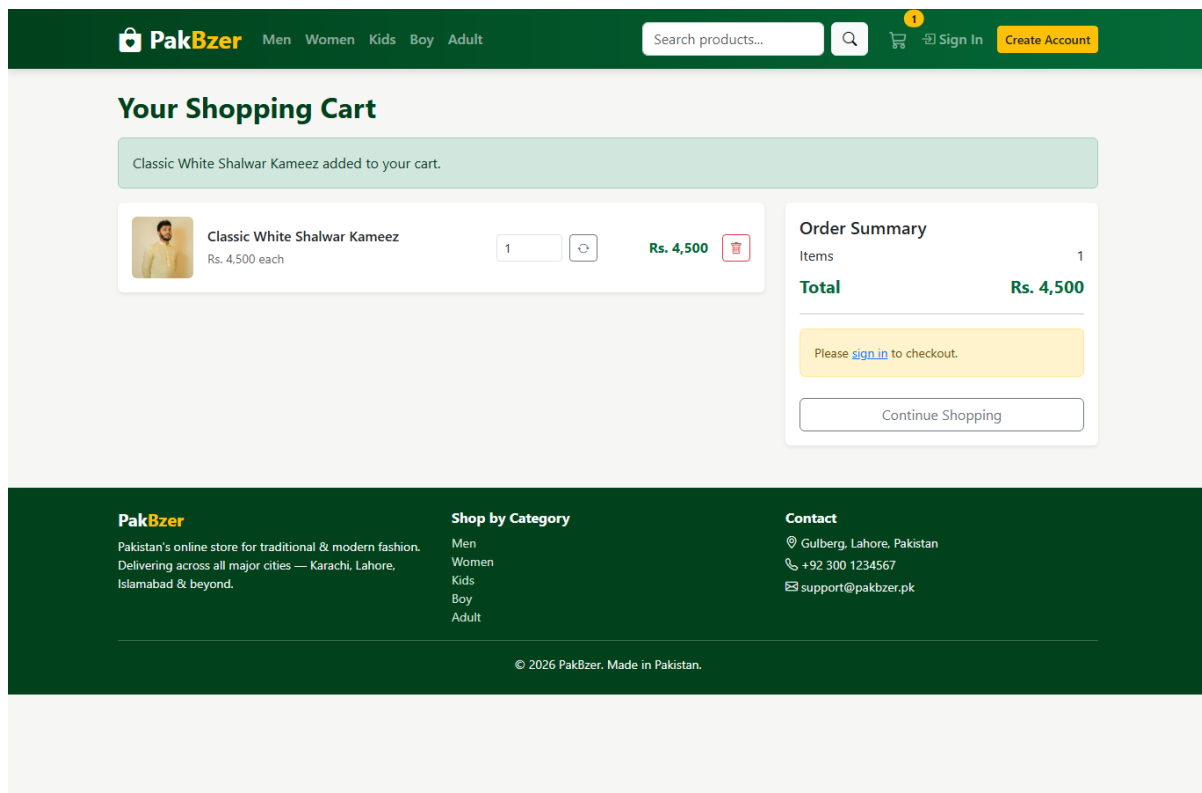


Figure 5. Cart after adding one product.

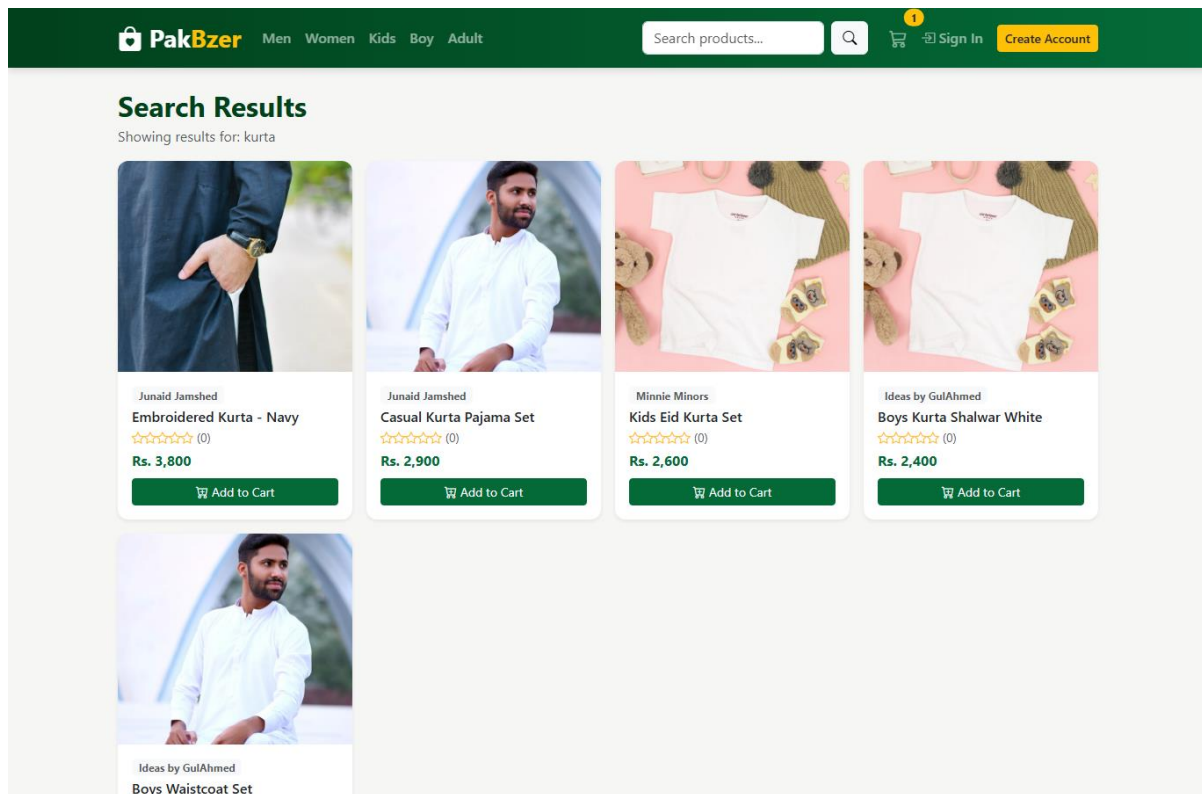


Figure 6. Search results for the keyword 'kurta'.

Results and Testing

Build Verification

The project was verified with Maven on Java 17. The command below completed successfully after compiling resources and Java source files. Maven reported that there were no test source files in src/test.

```
mvn test
```

```
[INFO] Building PakBzer 1.0.0
[INFO] No tests to run.
[INFO] BUILD SUCCESS
```

Manual Browser Testing

Test Case	Expected Result	Observed Result
Open /	Home page loads with category tiles and previews.	Passed. Page returned HTTP 200 and rendered product sections.
Open /category/women	Women's category lists products.	Passed. Page showed 10 products.
Open /product/1	Product details, price, rating,	Passed. Product detail rendered

	<i>and add-to-cart form appear.</i>	<i>correctly.</i>
<i>Add product to cart</i>	<i>Cart shows the item, quantity, and total price.</i>	<i>Passed. Cart count became 1 and total was Rs. 4,500.</i>
<i>Open /search?q=kurta</i>	<i>Matching products appear.</i>	<i>Passed after correcting a malformed Thymeleaf text attribute in search.html.</i>
<i>Open /login</i>	<i>Login form and Google OAuth option appear.</i>	<i>Passed. Login page rendered correctly.</i>

Table 4. Manual testing results.

Conclusion and Future Work

PakBzer successfully demonstrates how Object-Oriented Programming principles can be used in a complete web application. The project models real store concepts through Java classes, applies inheritance through BaseEntity, encapsulates important calculations and cart behavior, abstracts product access through a service interface, and uses polymorphism through Spring-managed implementations and overridden security behavior.

The final application supports the core e-commerce workflow: browsing, searching, product viewing, cart management, login/registration, reviews, and checkout integration. Its layered design makes the codebase readable and suitable for future extension.

Future Work

- *Add an admin dashboard for product, inventory, category, and order management.*
- *Add persistent orders, order history, invoices, and delivery tracking.*
- *Replace H2 with MySQL or PostgreSQL for production deployment.*
- *Add automated unit, integration, and controller tests.*
- *Move all OAuth and Stripe secrets fully into environment variables before public deployment.*
- *Add Stripe webhook handling and payment status verification.*
- *Improve search with filters for price, brand, category, rating, and stock.*
- *Deploy the application to a cloud server with HTTPS and production security settings.*

References

[1] Spring. Spring Boot 3.2.5 Reference Documentation. <https://docs.spring.io/spring-boot/docs/3.2.5/reference/htmlsingle/> Accessed June 11, 2026.

[2] Spring. Spring Security Reference. <https://docs.spring.io/spring-security/reference/index.html> Accessed June 11, 2026.

[3] Thymeleaf. Thymeleaf Documentation and Spring Integration. <https://www.thymeleaf.org/documentation.html> Accessed June 11, 2026.

[4] H2 Database Engine. Official Documentation. <https://www.h2database.com/> Accessed June 11, 2026.

[5] Stripe. Stripe API and SDK Documentation. <https://docs.stripe.com/api> and <https://docs.stripe.com/sdks> Accessed June 11, 2026.

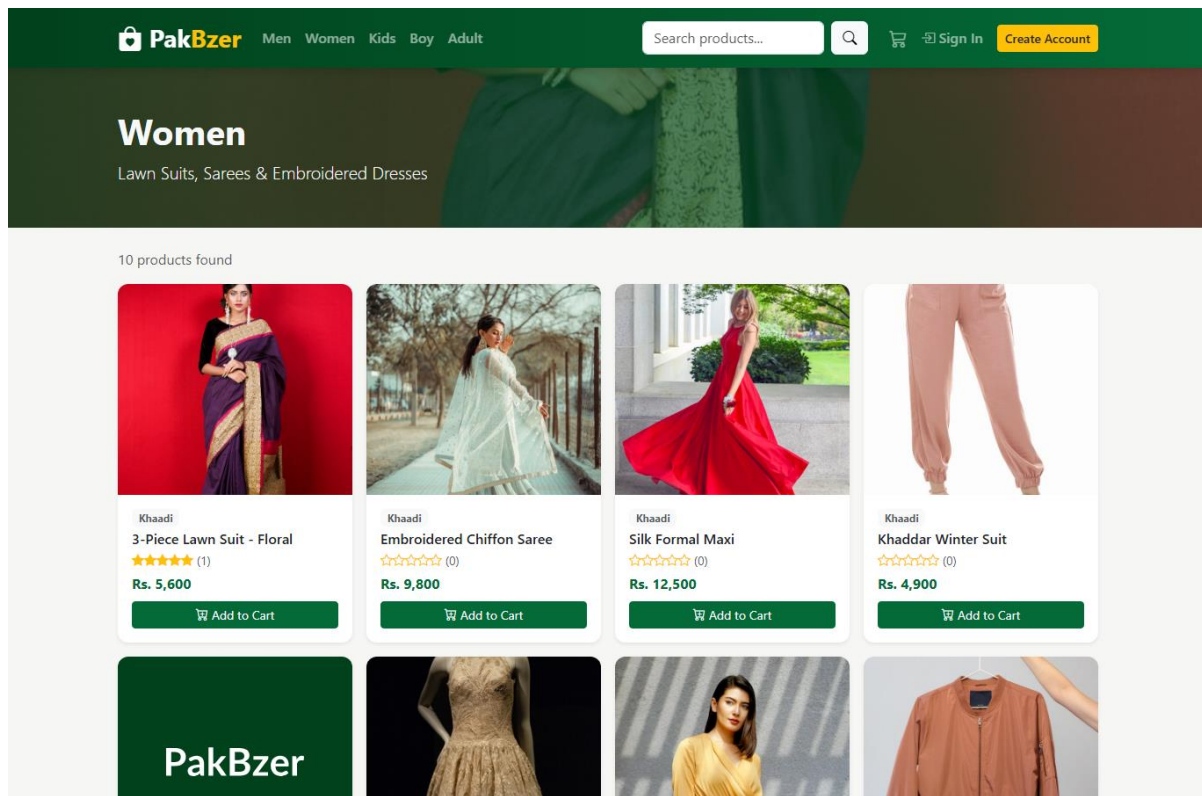
[6] Bootstrap. Bootstrap 5 Documentation. <https://getbootstrap.com/docs/5.0/getting-started/introduction/> Accessed June 11, 2026.

[7] Oracle. JDK 17 Documentation. <https://docs.oracle.com/en/java/javase/17/> Accessed June 11, 2026.

[8] Apache Maven. POM Reference. <https://maven.apache.org/pom.html> Accessed June 11, 2026.

Appendices

Appendix A: Additional Screenshots



Appendix Figure A1. Women's category product listing.

PakBzer Men Women Kids Boy Adult

Search products...

1 Sign In Create Account

Sign In to PakBzer

Email

Password

Sign In

— or —

Continue with Google

New here? [Create an account](#)

PakBzer
Pakistan's online store for traditional & modern fashion.
Delivering across all major cities — Karachi, Lahore,
Islamabad & beyond.

Shop by Category
Men
Women
Kids
Boy
Adult

Contact
Gulberg, Lahore, Pakistan
+92 300 1234567
support@pakbzer.pk

© 2026 PakBzer. Made in Pakistan.

Appendix Figure A2. Login page with local and Google sign-in options.

Appendix B: Build and Run Commands

```
# Run tests / compile verification
mvn test

# Start the Spring Boot application
mvn spring-boot:run

# Local application URL
http://localhost:8080

# H2 console
http://localhost:8080/h2-console
JDBC URL: jdbc:h2:file:./data/pakbzer
User: sa
Password: <blank>
```

Appendix C: Important Source Files

<i>File</i>	<i>Purpose</i>
<i>BaseEntity.java</i>	<i>Abstract inheritance root for id and audit timestamps.</i>
<i>Product.java</i>	<i>Product domain model with rating and stock behavior.</i>
<i>Cart.java</i>	<i>Session-scoped cart with add, update, remove, and total logic.</i>
<i>ProductService.java</i>	<i>Service interface for product catalog abstraction.</i>
<i>ProductServiceImpl.java</i>	<i>Concrete product catalog service implementation.</i>
<i>SecurityConfig.java</i>	<i>Form login, OAuth2 login, route authorization, and logout configuration.</i>
<i>DataSeeder.java</i>	<i>Seeds the local H2 database with Pakistani fashion products.</i>
<i>templates/*.html</i>	<i>Thymeleaf pages for home, product, category, cart, login, search, and checkout.</i>

Appendix Table C1. Source file summary.